

Concurrency Control

Dr. Geetha Mary A
Professor,
SCOPE, VIT

Source:

Pearson Education, Inc. 2011, Elmasri/Navathe, Fundamentals of Database Systems, Sixth Edition

Introduction to Transaction Processing – Transaction concepts: ACID Properties of Transactions, Transaction States - Serial and Serializable Schedules - Schedules based on recoverability – Schedules based on Serializability - Conflict Serializability - Recovery Concepts: Log Based Recovery Protocols, Recovery based on deferred update, Recovery techniques based on immediate update – Shadow Paging Algorithm		
Module:6	Concurrency Control In Transaction Processing	8 hours
Concurrent Transactions – Lost Update Problem - Concurrency Control Techniques: Time Stamp Based Protocols, Thomas Write Rule, Lock Based Protocols, Lock Compatibility Matrix, - Two-Phase Locking Protocol - Lock Conversions - Graph Based Protocols for Concurrency Control - Tree Protocol for Concurrency Control – Deadlocks Based on Locks in Transactions – Deadlock Handling Techniques – Transaction Deadlock Detection Techniques – Transaction Deadlock Prevention Techniques – Multi-Granularity Locking for avoiding Transaction Deadlocks		
Module:7	NOSQL Database Management	3 hours
Introduction, Need of NoSQL, CAP Theorem, different NoSQL data bases: Key-value data stores, Columnar families, Document databases, Graph databases		
Module:8	Contemporary Issues	2 Hours
	Total Lecture hours:	45 hours

Database Concurrency Control

1 Purpose of Concurrency Control

- To enforce Isolation (through mutual exclusion) among conflicting transactions.
- To preserve database consistency through consistency preserving execution of transactions.
- To resolve read-write and write-write conflicts.

Example: In concurrent execution environment if T1 conflicts with T2 over a data item A, then the existing concurrency control decides if T1 or T2 should get the A and if the other transaction should wait.

Database Concurrency Control

Why Concurrency Control is needed:

- **The Lost Update Problem.**

This occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database item incorrect.

- **The Temporary Update (or Dirty Read) Problem.**

This occurs when one transaction updates a database item and then the transaction fails for some reason. The updated item is accessed by another transaction before it is changed back to its original value.

Database Concurrency Control

Why Concurrency Control is needed (cont.):

- **The Incorrect Summary Problem .**

If one transaction is calculating an aggregate summary function on a number of records while other transactions are updating some of these records, the aggregate function may calculate some values before they are updated and others after they are updated.

FIGURE 17.3

Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem.

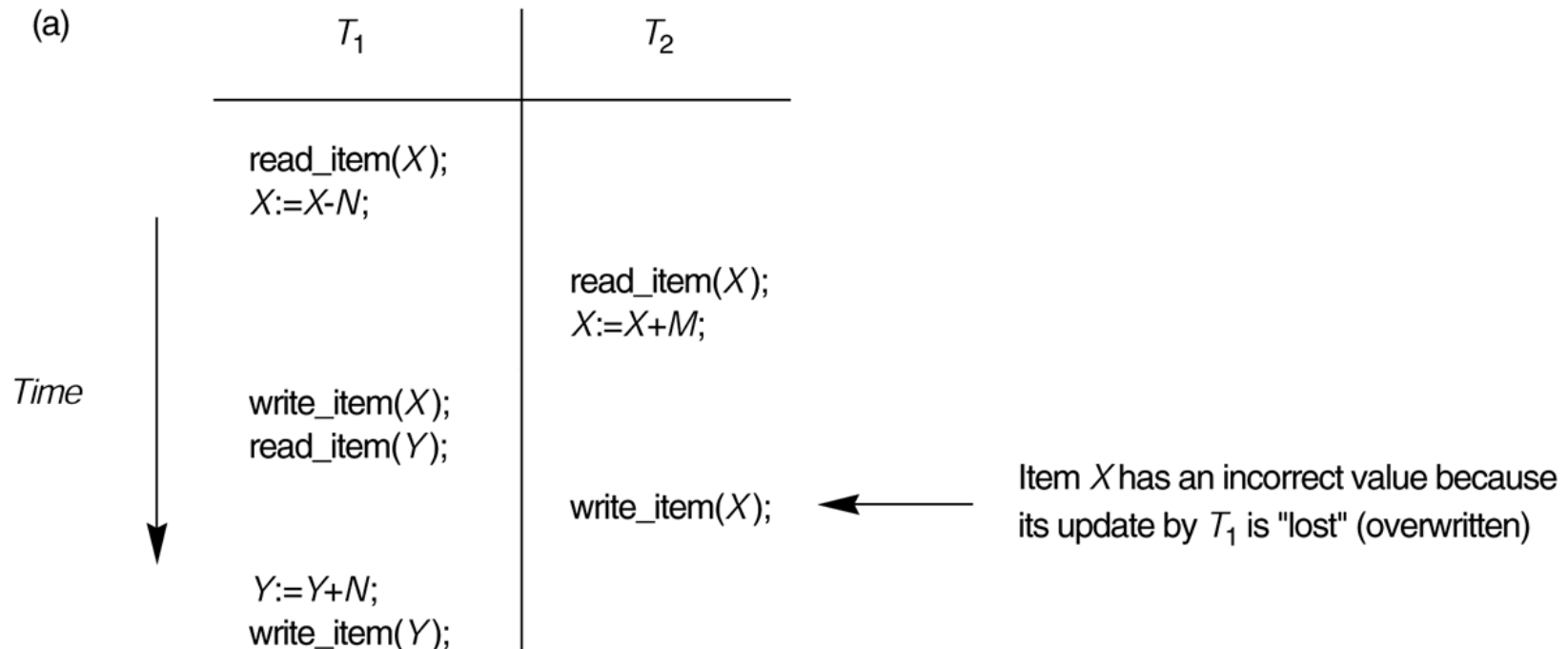


FIGURE 17.3 (continued)

Some problems that occur when concurrent execution is uncontrolled. (b) The temporary update problem.

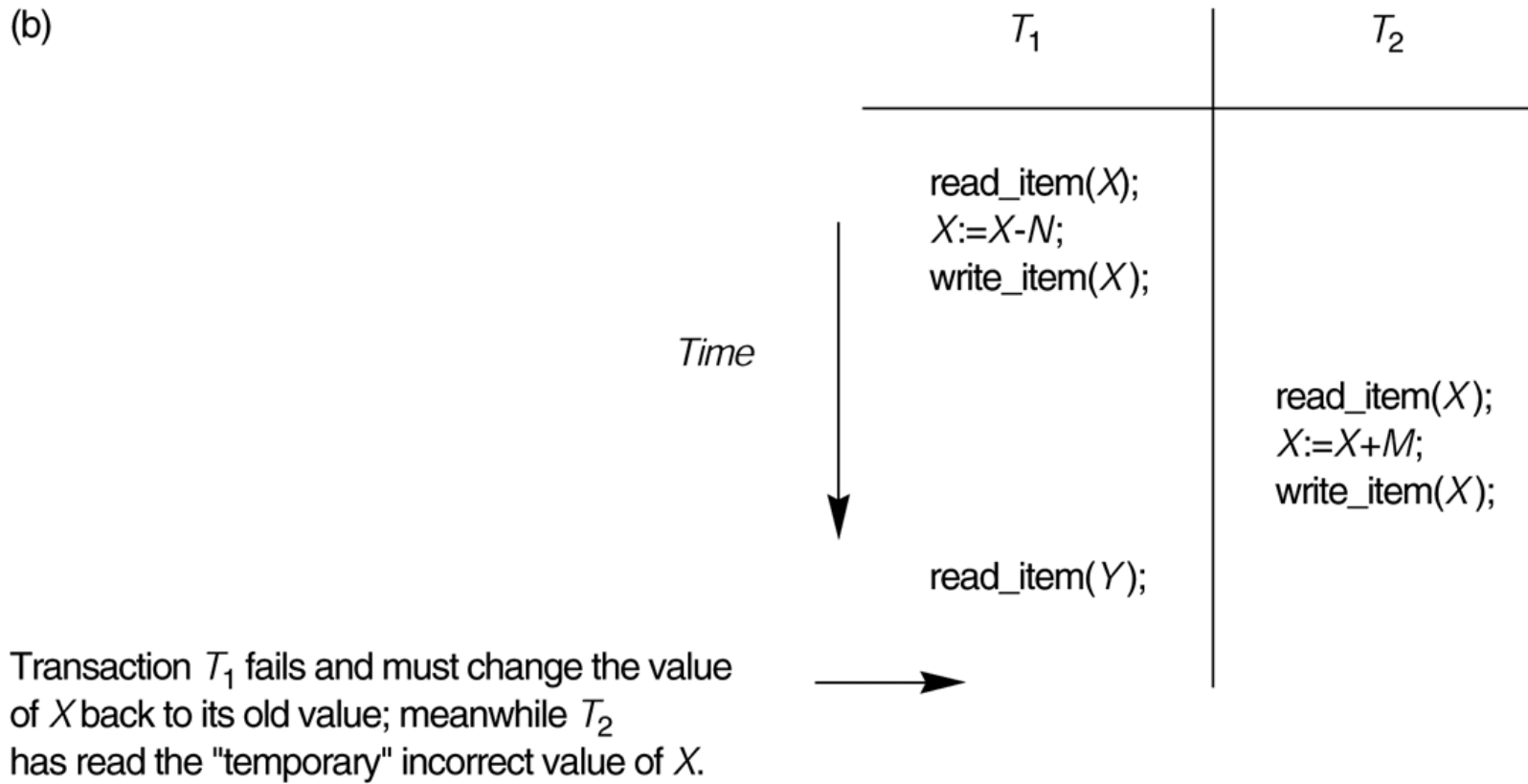
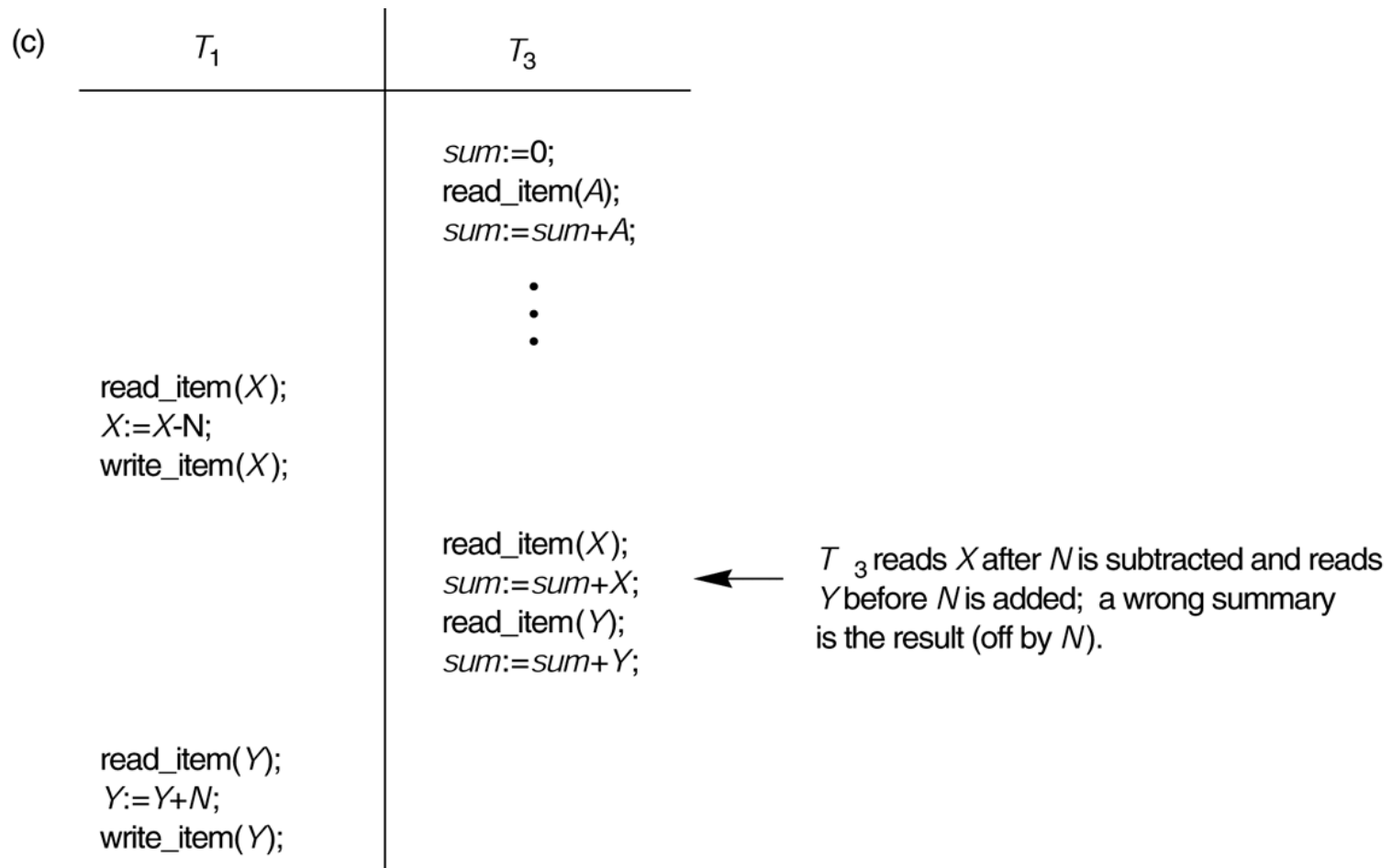


FIGURE 17.3 (continued)

Some problems that occur when concurrent execution is uncontrolled. (c) The incorrect summary problem.



Database Concurrency Control

Locking Techniques: Binary Locking Scheme

A binary lock can have two states or values: locked (1) or unlocked (0)

If the value of the lock on item X is 1, item X cannot be accessed by a database operation that requests X.

If the value of the lock on item X is 0, item X can be accessed when requested.

The current value of the lock on item X is referred to as LOCK(X).

Binary locks are kept in a lock table which contains records where each record contains three fields:

<DB Item, LOCK, Locking Transaction>

Only items that are locked are kept in the table. Items that are not in the table are considered to be unlocked.

Database Concurrency Control

Locking Techniques: Binary Lock Operations

The following code performs the lock operation:

lock-item(X):

```
B: if LOCK (X) = 0 (*item is unlocked*)  
    then LOCK (X)  $\leftarrow$  1 (*lock the item*)  
    else begin  
        wait (until lock (X) = 0 and  
            the lock manager wakes up the transaction);  
    goto B  
end;
```

Database Concurrency Control

Locking Techniques: Binary Lock Operations

The following code performs the unlock operation:

unlock-item(X):

LOCK (X) $\leftarrow$ 0 (*unlock the item*)

if any transactions are waiting then

 wake up one of the waiting the transactions;

Database Concurrency Control

Locking Techniques: Essential components

To use the binary locking scheme, every transaction must obey the following rules:

- A transaction T must issue the operation `lock_item(X)` before any `read_item(X)` or `write_item(X)` operations are performed in T.
- A transaction T must issue the operation `unlock_item(X)` after all `read_item(X)` or `write_item(X)` operations are completed in T.
- A transaction T will not issue a `lock_item(X)` operation if it already holds the lock on item X.
- A transaction T will not issue a `unlock_item(X)` unless it already holds the lock on item X.

Database Concurrency Control

Locking Techniques: Shared/Exclusive Locking Scheme

Two locks modes (a) shared (read) and (b) exclusive (write).

Shared mode: shared lock (X). More than one transaction can apply shared lock on X for reading its value but no write lock can be applied on X by any other transaction.

Exclusive mode: Write lock (X). Only one write lock on X can exist at any time and no shared lock can be applied by any other transaction on X.

LOCK(X) has now 3 states: “read_locked”, “write_locked” or “unlocked”.

Conflict matrix

	Read	Write
Read	N	Y
Write	Y	Y

Database Concurrency Control

Locking Techniques: Shared/Exclusive Lock Operations

The following code performs the read lock operation:

read_lock(X):

B: if LOCK (X) = “unlocked” then

begin LOCK (X) $\leftarrow$ “read-locked”;

no_of_reads (X) $\leftarrow$ 1;

end

else if LOCK (X) $\leftarrow$ “read-locked” then

no_of_reads (X) $\leftarrow$ no_of_reads (X) + 1

else begin wait (until LOCK (X) = “unlocked” and
the lock manager wakes up the transaction);

go to B

end;

Database Concurrency Control

Locking Techniques: Shared/Exclusive Lock Operations

The following code performs the write lock operation:

write_lock(X):

B: if LOCK (X) = “unlocked” then

 LOCK (X) $\leftarrow$ “write-locked”;

 else begin

 wait (until LOCK (X) = “unlocked” and

 the lock manager wakes up the transaction);

 go to B

 end;

Database Concurrency Control

Locking Techniques: Shared/Exclusive Lock Operations

The following code performs the unlock operation:

unlock(X):

```
if LOCK (X) = "write-locked" then
    begin LOCK (X) ← "unlocked";
        wakeup one of the transactions, if any
    end
else if LOCK (X) ← "read-locked" then
    begin
        no_of_reads (X) ← no_of_reads (X) -1
        if no_of_reads (X) = 0 then
            begin
                LOCK (X) = "unlocked";
                wakeup one of the transactions, if any
            end
        end
    end
end;
```

To use the Shared/Exclusive locking scheme, every transaction must obey the following rules:

- A transaction T must issue the operation `read_lock(X)` or `write_lock(X)` before any `read_item(X)` operation is performed in T.
- A transaction T must issue the operation `write_lock(X)` before any `write_item(X)` operation is performed in T.
- A transaction T must issue the operation `unlock(X)` after all `read_item(X)` or `write_item(X)` operations are completed in T.
- A transaction T will not issue a `read_lock(X)` operation if it already holds a read (shared) or a write (exclusive) lock on item X. This rule may be relaxed.
- A transaction T will not issue a `write_lock(X)` operation if it already holds a read (shared) or a write (exclusive) lock on item X. This rule may be relaxed.
- A transaction T will not issue a `unlock_item(X)` unless it already holds a read (shared) or a write (exclusive) lock on item X.

Database Concurrency Control

Locking Techniques: Essential components

Lock conversion

Lock upgrade: existing read lock to write lock

if T_i has a read-lock (X) and T_j has no read-lock (X) ($i \neq j$) then
 convert read-lock (X) to write-lock (X)
else
 force T_i to wait until T_j unlocks X

Lock downgrade: existing write lock to read lock

T_i has a write-lock (X) (*no transaction can have any lock on X*)
 convert write-lock (X) to read-lock (X)

Database Concurrency Control

Locking Techniques: Using Shared/Exclusive Locking (Serial Transactions)

T1

read_lock (Y);
read_item (Y);
unlock (Y);
write_lock (X);
read_item (X);
X:=X+Y;
write_item (X);
unlock (X);

T2

read_lock (X);
read_item (X);
unlock (X);
Write_lock (Y);
read_item (Y);
Y:=X+Y;
write_item (Y);
unlock (Y);

Result

Initial values: X=20; Y=30
Result of serial execution
T1 followed by T2
X=50, Y=80.
Result of serial execution
T2 followed by T1
X=70, Y=50

Database Concurrency Control

Locking Techniques: Using Shared/Exclusive Locking

(Interleaved Transactions)

Time
↓

T1

read_lock (Y);
read_item (Y);
unlock (Y);

write_lock (X);
read_item (X);
X:=X+Y;
write_item (X);
unlock (X);

T2

read_lock (X);
read_item (X);
unlock (X);
write_lock (Y);
read_item (Y);
Y:=X+Y;
write_item (Y);
unlock (Y);

Result

X=50; Y=50
Nonserializable.

Database Concurrency Control

Two-Phase Locking Techniques: The algorithm

Two Phases: (a) Locking (Growing) (b) Unlocking (Shrinking).

Locking (Growing) Phase: A transaction applies locks (read or write) on desired data items one at a time.

Unlocking (Shrinking) Phase: A transaction unlocks its locked data items one at a time.

Requirement: For a transaction these two phases must be mutually exclusively, that is, during locking phase unlocking phase must not start and during unlocking phase locking phase must not begin.

Database Concurrency Control

Two-Phase Locking Techniques: The algorithm

T1'

read_lock (Y);
read_item (Y);
write_lock (X);
unlock (Y);
read_item (X);
X:=X+Y;
write_item (X);
unlock (X);

T2'

read_lock (X);
read_item (X);
write_lock (Y);
unlock (X);
read_item (Y);
Y:=X+Y;
write_item (Y);
unlock (Y);

T1' and T2' follow two-phase policy but they are subject to deadlock, which must be dealt with.

Database Concurrency Control

Variations of Two-Phase Locking Algorithms:

Conservative: Prevents deadlock by locking **all desired data items before transaction begins** execution by predeclaring its read_set and write_set.

Basic: Transaction locks data items **incrementally**. This may cause deadlock which is dealt with.

Strict: This is the most commonly used two-phase locking algorithm. A transaction T does not release any of its **exclusive (write) locks** until after it commits or aborts. Hence, no other transaction can read or write an item that is written by T unless T has committed.

Rigorous: More restrictive than Strict. A transaction does not release any of its locks (**exclusive or shared**) until after it commits or aborts.

Database Concurrency Control

Dealing with Deadlock and Starvation

Deadlock

T1'

read_lock (Y);
read_item (Y);

write_lock (X);
(waits for X)

T2'

read_lock (X);
read_item (X);

write_lock (Y);
(waits for Y)

T1' and T2' did follow two-phase
policy but they are deadlock

Deadlock (T'1 and T'2)

Database Concurrency Control

Dealing with Deadlock and Starvation

Deadlock Prevention

Other deadlock prevention algorithms use the concept of a timestamp $TS(T)$ which is a unique identifier assigned to transactions based on the order in which they start. If T_1 starts before T_2 , then

$TS(T_1) < TS(T_2)$ (older transaction has the smaller timestamp value)

- Suppose that transaction T_i tries to lock an item X but is not able to because X is locked by some other transaction T_j with a conflicting lock.

Examples of such algorithms are:

Wait-die: If $TS(T_i) < TS(T_j)$, then (T_i is older than T_j) **T_i is allowed to wait**; otherwise (T_i younger than T_j) abort T_i (T_i dies) and restart it later with the same timestamp.

Wound-wait: If $TS(T_i) < TS(T_j)$, then (T_i is older than T_j) **abort T_j** (T_i wounds T_j) and restart it later with the same timestamp; otherwise (T_i younger than T_j) T_i is allowed to wait.

Database Concurrency Control

Dealing with Deadlock and Starvation

Deadlock Detection and Resolution

In this approach, deadlocks are allowed to happen. The scheduler maintains a wait-for-graph for detecting cycle. If a cycle exists, then one transaction involved in the cycle is selected (victim) and rolled-back.

Another approach to deal with deadlock is the use of timeouts. In this method, if a transaction waits for a period of time longer than a system-defined timeout period, the system assumes that a transaction maybe in state of deadlock and aborts it regardless of whether a deadlock actually exists or not.

Database Concurrency Control

Dealing with Deadlock and Starvation

- **Deadlock avoidance**

- There are many variations of two-phase locking algorithm.
- Some avoid deadlock by not letting the cycle to complete.
- That is as soon as the algorithm discovers that blocking a transaction is likely to create a cycle, it rolls back the transaction.
- Wound-Wait and Wait-Die algorithms use timestamps to avoid deadlocks by rolling-back victim.

Database Concurrency Control

Dealing with Deadlock and Starvation

- **Starvation**

- Starvation occurs when a particular transaction consistently waits or restarted and never gets a chance to proceed further.
- In a deadlock resolution it is possible that the same transaction may consistently be selected as victim and rolled-back.
- This limitation is inherent in all priority based scheduling mechanisms.
- In Wound-Wait scheme a younger transaction may always be wounded (aborted) by a long running older transaction which may create starvation.

Database Concurrency Control

Timestamp based concurrency control algorithm

- **Timestamp**

- A monotonically increasing variable (integer) indicating the age of an operation or a transaction. A larger timestamp value indicates a more recent event or operation.
- Timestamp based algorithm uses timestamp to serialize the execution of concurrent transactions.

Database Concurrency Control

Timestamp based concurrency control algorithm

- **Basic Timestamp Ordering**

- 1. Transaction T issues a `write_item(X)` operation:
 - If $\text{read_TS}(X) > \text{TS}(T)$ or if $\text{write_TS}(X) > \text{TS}(T)$, then an younger transaction has already read the data item so abort and roll-back T and reject the operation.
 - If the condition in part (a) does not exist, then execute `write_item(X)` of T and set $\text{write_TS}(X)$ to $\text{TS}(T)$.
- 2. Transaction T issues a `read_item(X)` operation:
 - If $\text{write_TS}(X) > \text{TS}(T)$, then an younger transaction has already written to the data item so abort and roll-back T and reject the operation.
 - If $\text{write_TS}(X) \leq \text{TS}(T)$, then execute `read_item(X)` of T and set $\text{read_TS}(X)$ to the larger of $\text{TS}(T)$ and the current $\text{read_TS}(X)$.

Database Concurrency Control

Timestamp based concurrency control algorithm

- **Strict Timestamp Ordering**

- 1. Transaction T issues a `write_item(X)` operation:
 - If $TS(T) > read_TS(X)$, then delay T until the transaction T' that wrote or read X has terminated (committed or aborted).
- 2. Transaction T issues a `read_item(X)` operation:
 - If $TS(T) > write_TS(X)$, then delay T until the transaction T' that wrote or read X has terminated (committed or aborted).

Database Concurrency Control

Timestamp based concurrency control algorithm

- **Thomas's Write Rule**

- If $\text{read_TS}(X) > \text{TS}(T)$ then abort and roll-back T and reject the operation.
- If $\text{write_TS}(X) > \text{TS}(T)$, then just ignore the write operation and continue execution. This is because the most recent writes counts in case of two consecutive writes.
- If the conditions given in 1 and 2 above do not occur, then execute $\text{write_item}(X)$ of T and set $\text{write_TS}(X)$ to $\text{TS}(T)$.

Database Concurrency Control

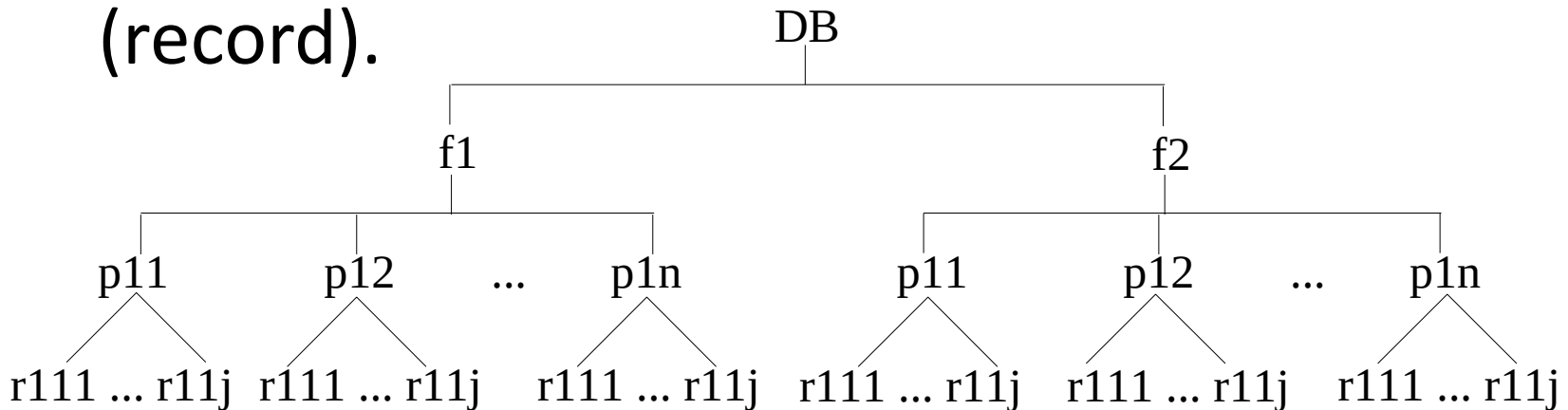
Granularity of data items and Multiple Granularity Locking

- A lockable unit of data defines its granularity. Granularity can be coarse (entire database) or it can be fine (a tuple or an attribute of a relation).
- Data item granularity significantly affects concurrency control performance. Thus, the degree of concurrency is low for coarse granularity and high for fine granularity.
- Example of data item granularity:
 1. A field of a database record (an attribute of a tuple)
 2. A database record (a tuple or a relation)
 3. A disk block
 4. An entire file
 5. The entire database

Database Concurrency Control

Granularity of data items and Multiple Granularity Locking

- The following diagram illustrates a hierarchy of granularity from coarse (database) to fine (record).



Database Concurrency Control

Granularity of data items and Multiple Granularity Locking

- To manage such hierarchy, in addition to read and write, three additional locking modes, called intention lock modes are defined:
 - **Intention-shared (IS)**: indicates that a shared lock(s) will be requested on some descendent nodes(s).
 - **Intention-exclusive (IX)**: indicates that an exclusive lock(s) will be requested on some descendent node(s).
 - **Shared-intention-exclusive (SIX)**: indicates that the current node is locked in shared mode but an exclusive lock(s) will be requested on some descendent nodes(s).

Database Concurrency Control

Granularity of data items and Multiple Granularity Locking

- These locks are applied using the following compatibility matrix:

	IS	IX	S	SIX	X
IS	yes	yes	yes	yes	no
IX	yes	yes	no	no	no
S	yes	no	yes	no	no
SIX	yes	no	no	no	no
X	no	no	no	no	no

Intention-shared (IS)

Intention-exclusive (IX)

Shared-intention-exclusive (SIX)

Database Concurrency Control

Granularity of data items and Multiple Granularity Locking

- The set of rules which must be followed for producing serializable schedule are
 1. The lock compatibility must adhered to.
 2. The root of the tree must be locked first, in any mode..
 3. A node N can be locked by a transaction T in S or IX mode only if the parent node is already locked by T in either IS or IX mode.
 4. A node N can be locked by T in X, IX, or SIX mode only if the parent of N is already locked by T in either IX or SIX mode.
 5. T can lock a node only if it has not unlocked any node (to enforce 2PL policy).
 6. T can unlock a node, N, only if none of the children of N are currently locked by T.

Database Concurrency Control

Granularity of data items and Multiple Granularity Locking: An example of a serializable execution:

T1	T2	T3
IX(db)		
IX(f1)		
	IX(db)	
		IS(db)
		IS(f1)
		IS(p11)
IX(p11)		
X(r111)		
	IX(f1)	
	X(p12)	
		S(r11j)
IX(f2)		
IX(p21)		
IX(r211)		
Unlock (r211)		
Unlock (p21)		
Unlock (f2)		
		S(f2)

Database Concurrency Control

- Granularity of data items and Multiple Granularity Locking: An example of a serializable execution (continued):

T1

unlock(r111)
unlock(p11)
unlock(f1)
unlock(db)

T2

unlock(p12)
unlock(f1)
unlock(db)

T3

unlock (r111j)
unlock (p11)
unlock (f1)
unlock(f2)
unlock(db)